



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

EXPLORANDO LA HUELLA LINGÜÍSTICA OCULTA EN LOS
MODELOS DE PROCESAMIENTO DEL LENGUAJE NATURAL

TESIS

QUE PARA OBTENER EL TÍTULO DE:

LICENCIADO EN CIENCIAS DE LA COMPUTACIÓN

PRESENTA:

JOSÉ LUIS ONOFRE FRANCO

TUTOR

MARÍA XIMENA GUTIÉRREZ VASQUES

2025



Contenidos

1	Introducción	3
2	Marco teórico	4
2.1	La inteligencia artificial y el procesamiento de lenguaje natural	4
2.1.1	Grandes Modelos de Lenguaje	5
2.2	Tokenización a nivel subpalabra	5
2.2.1	Codificación de Pares de Bytes	6
2.2.2	Otros métodos estadísticos de tokenización	7
2.3	La liga entre la lingüística y los métodos computacionales actuales	7
2.3.1	Clasificación de las lenguas	8
2.3.2	Relación entre tokenizadores y las lenguas	8
2.4	Bases de datos lingüísticas	10
2.4.1	WALS	10
2.4.2	Grambank	10
2.4.3	Lang2vec	10
2.5	Relación entre espacios	11
2.5.1	Índice Rand	11
2.5.2	Índice Rand Ajustado	11
2.5.3	K-medias	11
3	Metodología	12
3.1	Datos/Corpus de entrenamiento multilingües	12
3.2	Procesamiento computacional de las bases de datos lingüísticas	13
3.2.1	WALS	13
3.2.2	Grambank	14
3.2.3	lang2vec	16
3.3	Base de referencia	16
3.4	Comparación entre los espacios	16
3.4.1	Experimentos	16
3.4.2	BPE vs WALS+Grambank	16
3.4.3	BPE vs WALS	17
3.4.4	BPE vs Grambank	17
3.4.5	Grambank vs WALS	17
3.4.6	Grambank vs lang2Vec	17
4	Resultados	18
4.1	BPE vs WALS+Grambank	18
4.2	BPE vs WALS	19
4.3	BPE vs Grambank	19
4.4	Grambank vs WALS	20
4.5	Grambank vs Lang2Vec	21
5	Discusión	23
6	Conclusiones	24
	Bibliografía	25
7	Apéndice	28
7.1	Conjunto de lenguas utilizadas	28
7.2	Conjunto de características utilizadas en Grambank	30

Capítulo 1

Introducción

El procesamiento del lenguaje natural (PLN) ha transformado significativamente el área de la inteligencia artificial (IA), en parte, gracias a la creación de los grandes modelos de lenguaje (*Large Language Models*, LLMs) que dan pie a las aplicaciones generativas de texto. La capacidad de estas tecnologías para procesar y generar texto gramaticalmente correcto, semánticamente contextualizado, han abierto nuevas líneas de investigación sobre cómo las máquinas representan y procesan el lenguaje humano, desde una perspectiva tanto computacional como lingüística.

Puesto que este tipo de métodos son no supervisados y se basan en principios estadísticos de comprensión de cadenas, hay una opinión generalizada de que estas tecnologías no se basan en fundamentos lingüísticos. Sin embargo, estudios recientes, como el realizado por Gutierrez-Vasques et al., (2023), sugieren que BPE podría codificar información lingüística. Para demostrarlo, los autores llevaron a cabo un experimento que consistió en: (1) utilizar BPE para caracterizar las lenguas a partir de las propiedades de sus unidades subpalabra; (2) emplear bases de conocimiento lingüístico para representar las lenguas mediante vectores; y (3) comparar, mediante un algoritmo de agrupamiento, si las lenguas se organizan de forma similar en ambos espacios de representación. Los resultados mostraron que las características obtenidas por BPE son capaces de agrupar lenguas que comparten características lingüísticas similares. Lo que indica que este segmentador, aunque aparentemente superficial, puede capturar rasgos estructurales relevantes de las lenguas

No obstante, en su investigación no se consideró la comparación con bases de datos lingüísticas que concentran una amplia gama de lenguas y características gramaticales tipológicas, como Grambank. Este recurso, desarrollado por la Sociedad Max Planck, constituye una valiosa fuente de información, cuya integración reforzaría sustancialmente la hipótesis sobre la codificación lingüística del algoritmo BPE. Asimismo, el refinamiento en la caracterización vectorial de lenguas y la implementación de métodos de agrupación más sofisticados permitiría un análisis comparativo de mayor precisión, contribuyendo a una validación más robusta de los resultados obtenidos.

De este modo, se parte de algoritmos de segmentación preexistentes, como BPE, a los que se les incorpora un análisis minucioso de las características de las unidades subpalabra generadas, y se propone una metodología computacional para verificar que dichos patrones corresponden con las nociones descritas en estudios lingüísticos.

En suma, esta tesis busca avanzar en la comprensión del tipo de conocimiento lingüístico que las tecnologías del lenguaje actuales codifican de forma implícita, un objetivo clave en campos como la lingüística computacional y el PLN.

Capítulo 2

Marco teórico

2.1 La inteligencia artificial y el procesamiento de lenguaje natural

Los modelos de lenguaje han sido uno de los puntos de interacción entre la inteligencia artificial (IA) y el procesamiento del lenguaje natural (PLN). Esta intersección impulsó décadas de investigación y la confluencia de varias disciplinas. En el centro de todo ello, las palabras han sido el elemento fundamental sobre el que se construyó toda representación y generación del lenguaje. Para comprender el alcance de esta evolución, es necesario partir desde la propia naturaleza de los modelos de lenguaje.

Por tanto, los modelos de lenguaje son el resultado del PLN para representar y comprender el lenguaje humano. Así mismo, las formas de modelar el lenguaje natural han ido cambiando a lo largo del tiempo, teniendo influencia en los resultados de las tareas y sistemas de PLN.

En los primeros sistemas de PLN, el modelado del lenguaje natural se abordó mediante reglas gramaticales explícitas —denotables con gramáticas libres de contexto— con una fuerte influencia lingüística. Esto puede observarse desde una de las primeras demostraciones de traducción automática, el experimento Georgetown-IBM (Hutchins, 2004), que empleó solo 6 reglas gramaticales, hasta arquitecturas más elaboradas como EUROTRA¹, cuyo modelo de transferencia (Johnson et al., 1985; Varile & Lau, 1988) realizaba un análisis sintáctico y semántico de la lengua de origen para luego transferir dicha representación a la lengua destino. En ambos casos, estos enfoques requerían un intenso trabajo lingüístico para definir las reglas y construir un modelo de lenguaje funcional.

No obstante, los modelos de lenguaje estadísticos (*Statistical Language Model*, SLM) reemplazaron a los modelos basados en reglas al demostrar mejores resultados en tareas de NLP (Brown et al., 1990). En lugar de partir de un conjunto predefinido de reglas lingüísticas, los SLM infieren los patrones más probables del lenguaje mediante el conteo sobre grandes corpus de texto. Esto representó un cambio de paradigma en el desarrollo de los sistemas de PLN a formularse como problemas estadísticos.

De esta manera, un modelo de lenguaje define una probabilidad sobre una secuencia de palabras $w_{1:T} = (w_1, w_2, \dots, w_T)$ (Bengio et al., 2003)

$$P(w_{1:T}) = \prod_{t=1}^T P(w_t \mid w_{1:t-1}) \quad (1)$$

donde cada w_i pertenece a un vocabulario V . De esta manera, la probabilidad de $w_{1:T}$ se puede descomponer en probabilidades condicionales, lo cual define los parámetros que el modelo tiene que aprender. Sin embargo, a medida que la secuencia de palabras incrementa, así los parámetros también.

En consecuencia, el problema se centró en aproximar el cálculo de estas probabilidades para los modelos de lenguaje. Es por esto que en los SLM predominó el uso de n-gramas, que

¹EUROTRA fue un proyecto de traducción automática establecido y financiado por la Comisión Europea desde 1978 hasta 1992.

son subsecuencias de N elementos o palabras. Los n-gramas usan la suposición de Markov (Markov, 2006), donde la suposición que cada probabilidad condicional sólo depende de los previos N términos y no de la secuencia entera. A partir de esta suposición se construye el modelo de n-gramas.

2.1.1 Grandes Modelos de Lenguaje

No obstante, los modelos de n-gramas enfrentan un problema fundamental: la maldición de la dimensionalidad (Bengio et al., 2003; Zhao et al., 2025) al querer usar contextos –valores de n – largos. Estimar con precisión modelos de lenguaje de alto orden resulta difícil y en consecuencia, requieren calcular un número exponencial de probabilidades de transición, lo cual aumenta su costo computacional.

Para resolver este problema, la investigación recurrió a los modelos de lenguaje basados en redes neuronales (*Neural Network Language Models*, NNLM). Este enfoque abandonó el conteo de palabras y, en su lugar, las representó como vectores en un espacio continuo (Bengio et al., 2003; Mikolov et al., 2013). Gracias a esto, el modelo logra generalizar mejor, pues puede identificar similitudes entre palabras con significados relacionados, incluso cuando estas no aparecen juntas con frecuencia en el corpus de entrenamiento.

Esta representación vectorial es fundamental en la arquitectura *Transformer* (Vaswani et al., 2017), la cual introdujo el mecanismo de atención para relacionar tokens del contexto sin importar su distancia en la secuencia, además de permitir el procesamiento en paralelo. Sobre esta base se desarrollaron modelos como BERT (Devlin et al., 2019) y GPT (Radford et al., 2019), entre otros.

Los Transformers generan estas representaciones vectoriales en el codificador (*encoder*). Aquí, cada token de la entrada se transforma en un vector a partir de lo aprendido durante el entrenamiento. Este vector se refina mediante el mecanismo de autoatención (*self-attention*) (Vaswani et al., 2017), que permite al modelo relacionar cada token con los demás tokens de la secuencia.

2.2 Tokenización a nivel subpalabra

Estos grandes modelos de lenguaje no operan directamente sobre la cadena de caracteres de un texto entero, sino que requieren una etapa previa de preprocesamiento. Este primer proceso es la tokenización, la cual consiste en segmentar el texto en unidades discretas denominadas tokens. Estos tokens son las unidades manejadas por los modelos de lenguaje y algoritmos de PLN. Sin embargo, cada algoritmo se beneficia diferentemente en cómo se representan los tokens. Estas unidades pueden corresponder a palabras, subpalabras, morfemas u otros tipos de segmentos.

Texto original: «En un lugar lejano»

Tokenización a nivel palabra: [En, un, lugar, lejano]

Tokenización a nivel carácter: [E, n, u, n, l, u, g, a, r, l, e, j, a, n, o]

Listado 1: Algunas formas de tokenización.

Un primer enfoque de cómo se pueden representar los tokens es la tokenización por palabras, aunque tiene limitaciones (Jurafsky & Martin, 2025) importantes. Por un lado, lenguas como el chino y el japonés no tienen espacios entre palabras, lo que dificulta la tokenización cuando

se asume que cada palabra está separada por espacios. Por otro lado, no hay forma de procesar palabras desconocidas (*Out of Vocabulary*, OOV), sin expandir constantemente el vocabulario. Esto es evidente en tareas como la traducción (Sennrich et al., 2016) de palabras raras y OOV, donde los mecanismos al nivel de palabra no son suficientes para lenguas que tienen procesos productivos para formar nuevas palabras con el paso del tiempo, lo cual es una fuerte limitante.

Para abordar estas limitaciones, una alternativa al uso de palabras como tokens es emplear subpalabras. Las subpalabras corresponden a palabras completas, cadenas arbitrarias o incluso morfemas, lo cual da entender que son unidades que tienen una longitud igual o más pequeña que una palabra.

Esta propiedad del tamaño de una subpalabra resulta fundamental cuando un modelo se enfrenta a palabras desconocidas. Si una palabra aparece en muy pocas instancias, un modelo presenta dificultades en aprender el significado de esta, lo que limita su capacidad de generalización. En cambio, cuando el modelo utiliza subpalabras, dispone de más evidencia distribuida a lo largo del corpus, pues es más probable que aparezcan estas unidades. Como resultado, los modelos basados en subpalabras logran un mejor manejo de palabras OOV (Jurafsky & Martin, 2025; Sennrich et al., 2016).

Debido a estas ventajas, el uso de subpalabras como tokens ha sido predominante en los modelos de lenguaje actuales.

Las subpalabras son los tokens que los LLMs usan para las tareas de NLP. La principal ventaja de estos es que lidian con las palabras OOV.

2.2.1 Codificación de Pares de Bytes

La codificación de pares de bytes (*Byte-Pair Encoding*, BPE) (Gage, 1994) es un algoritmo usado para generar subpalabras (Sennrich et al., 2016). Estas subpalabras son el resultado de la compresión que realiza BPE sobre los caracteres de textos de entrenamiento. BPE identifica y fusiona iterativamente los pares de caracteres más frecuentes para generar nuevos símbolos. Cada nuevo símbolo es generado en cada iteración, a la cual llamamos fusión. El resultado de la última fusión nos da el conjunto subpalabras resultantes.

La descripción de BPE es la siguiente:

1. Se obtienen los pares de símbolos $[a_i, b_j]$ y sus frecuencias $f([a_i, b_j])$
2. Se obtiene:

$$[a, b] = \underset{a_i b_j}{\operatorname{argm\acute{a}x}} \{ f([a_i, b_j]) : a_i, b_j \in \Sigma \} \quad (2)$$

3. Se hace el remplazo por el símbolo ab en cada palabra del vocabulario:

$$[a, b] \rightarrow ab \quad (3)$$

4. Se agrega el símbolo ab al alfabeto Σ y se repite el proceso hasta alcanzar un número predeterminado de operaciones.

El algoritmo revela un punto importante sobre BPE, que es que funciona de manera no supervisada. Una de las ventajas de este enfoque es que es posible aplicar este algoritmo sobre

diferentes textos así como diferentes lenguas. Por lo tanto, cada lengua puede dar un conjunto diferente de subpalabras.

De este modo, una vez que BPE generó el conjunto de subpalabras, la tokenización sobre un texto sigue un procedimiento similar al algoritmo anterior. Este algoritmo convierte un texto en una secuencia de tokens.

1. En primer lugar, el modelo segmenta el texto a nivel de caracteres.
2. A continuación, el modelo aplica de forma iterativa las reglas de reemplazo que BPE aprendió durante el entrenamiento.
3. Este proceso combina secuencias de caracteres según las reglas aprendidas y produce finalmente la representación del texto en subpalabras.

Como ejemplo del proceso de tokenización, considérese este texto:

Las supernovas estallan en galaxias lejanas.

Al aplicar un tokenizador ya entrenado, en este caso tiktoken, el texto se convierte en una serie de tokens:

Las supernovas estallan en galaxias lejanas.

Cuando tenemos un modelo de BPE entrenado, podemos observar subpalabras que son frecuentes en las palabras, como «ción» en terminación, disminución, adjunción; y subpalabras que no son frecuentes en las palabras pero si por sí solas, como «un», «los», entre otros.

Retomando el ejemplo anterior, la tokenización dio subpalabras como *Las*, *en*, que son palabras muy comunes en español. A su vez, otras subpalabras que forman parte como *as*, se pueden encontrar en otras palabras como *bananas*, *sábanas*, *personas*.

2.2.2 Otros métodos estadísticos de tokenización

Existen otros métodos estadísticos para generar subpalabras con enfoques similares a BPE. Uno de ellos es WordPiece, caracterizado por crear las fusiones entre símbolos mediante una función de verosimilitud.

WordPiece busca maximizar la verosimilitud entre los símbolos de entrenamiento. Sean s_1 , s_2 símbolos usados en el entrenamiento de un modelo de WordPiece, entonces se fusionan los símbolos de si:

$$\arg \max_{s_1, s_2} \frac{P(s_1, s_2)}{P(s_1)P(s_2)} \quad (4)$$

Este enfoque es diferente a BPE que sólo busca la máxima frecuencia. Sin embargo, WordPiece es considerado también un algoritmo voraz y no supervisado. Aunque esta similitud entre WordPiece y BPE no implica que sus subpalabras resultantes sean las mismas.

2.3 La liga entre la lingüística y los métodos computacionales actuales

Históricamente, la tokenización ha sido considerada un proceso sin relevancia lingüística. Sin embargo, al ser la etapa encargada de segmentar el texto en unidades que el modelo procesa directamente, puede tener una influencia considerable en el desempeño de los LLMs.

En particular, el rendimiento del algoritmo BPE puede variar según las características lingüísticas de la lengua que se procesa (Parra, 2024). Entre estas, dos rasgos de la tipología lingüística resultan especialmente relevantes: la tipología sintáctica, que organiza las palabras dentro de una oración, y la tipología morfológica, que determina cómo se construyen las palabras internamente.

2.3.1 Clasificación de las lenguas

En un extremo de esa clasificación morfológica están las lenguas aislantes, donde las palabras no cambian de forma. Esa invariabilidad no significa que la lengua sea simple: el trabajo que en otras lenguas hacen las terminaciones o los prefijos, aquí lo hacen el orden de las palabras y el contexto. El chino mandarín es el ejemplo más conocido de este principio.

Un principio opuesto rige a las lenguas aglutinantes. En estas, las palabras crecen al incorporar fragmentos uno tras otro, cada uno con un significado propio y reconocible. Esos fragmentos se apilan de forma ordenada, casi como bloques ensamblados, lo que permite comprimir en una sola palabra lo que el español expresaría en varias. El turco funciona así.

Algo similar ocurre en las lenguas flexivas, aunque con una diferencia importante. Las palabras también se modifican, pero los fragmentos que se añaden no son tan fáciles de separar: una sola terminación puede expresar varias cosas al mismo tiempo. La -amos de «cantamos», por ejemplo, indica a la vez quién habla, cuántos son, en qué momento ocurre y con qué intención. El español, junto con el latín y el ruso, pertenecen a este grupo.

2.3.2 Relación entre tokenizadores y las lenguas

De manera similar, el tokenizador utilizado puede influir en el rendimiento de ciertas tareas de traducción dependiendo de las lenguas involucradas (Domingo et al., 2019). Asimismo, los tokenizadores cuyas subpalabras se alinean mejor con las unidades lingüísticas de una lengua tienden a mejorar el rendimiento de los modelos (Bayram et al., 2025). Esto sugiere que existe una relación entre los métodos de tokenización y las características de la lengua que se procesan.

Una de estas relaciones fue observada por Gutierrez-Vasques et al. (2023). Ellos definieron medidas para caracterizar a las lenguas de acuerdo a las propiedades de sus subpalabras generadas por BPE. Estas medidas se definen en base a un modelo entrenado de BPE, con el objetivo de ver si de alguna manera, las subpalabras codifican información lingüística relevantes para los modelos de lenguaje. Estas medidas, influenciadas por la tipología morfológica, son la *productividad* de una subpalabra, la *idiosincrasia* y la *frecuencia acumulada*. Tales medidas son calculadas a partir de las subpalabras que generó un modelo BPE, así como del corpus usado para obtener este modelo.

La medida de productividad está basada en la productividad lingüística. La productividad lingüística se refiere a cuán activamente se usa una regla gramatical para crear nuevas palabras o estructuras. Por ejemplo:

- El sufijo «-ble» en español es muy productivo: puede crear palabras como «comible», «bebible», «hackeable», «googleable».
- El sufijo «-idad» también es productivo: «amabilidad», «nacionalidad».

Se define la productividad de una subpalabra s como el número de palabras ortográficas que contienen a dicha subpalabra s en el corpus W :

$$\text{productividad}(s) = |W_s| \quad (5)$$

La frecuencia acumulada de una subpalabra s es la suma de las frecuencias de las palabras ortográficas que contienen a la subpalabra.

$$\text{c.freq}(s) = \sum_{w \in W_s} \text{freq}(w) \quad (6)$$

Mientras que la idiosincrasia está basada en la idiosincrasia lingüística. Esta última se refiere a las características particulares, irregulares o impredecibles de una lengua que no siguen patrones sistemáticos y deben aprenderse de manera individual. Por ejemplo:

- Plurales irregulares: «pie» → «pies» (regular), pero «menú» → «menús/menúes».
- Verbos irregulares: «ir» (voy, fui, iré) no sigue el patrón regular de los verbos.

La medida de idiosincrasia para una subpalabra s se define de la siguiente manera:

$$\text{idiosincrasia}(s) = \frac{\text{c.freq}(s)}{\text{productividad}(s)} \quad (7)$$

Gutierrez-Vasques et al. (2023) usaron estas medidas para caracterizar a 47 lenguas, con el cual crearon una representación vectorial para cada idioma. Podemos observar esto en Figura 1.

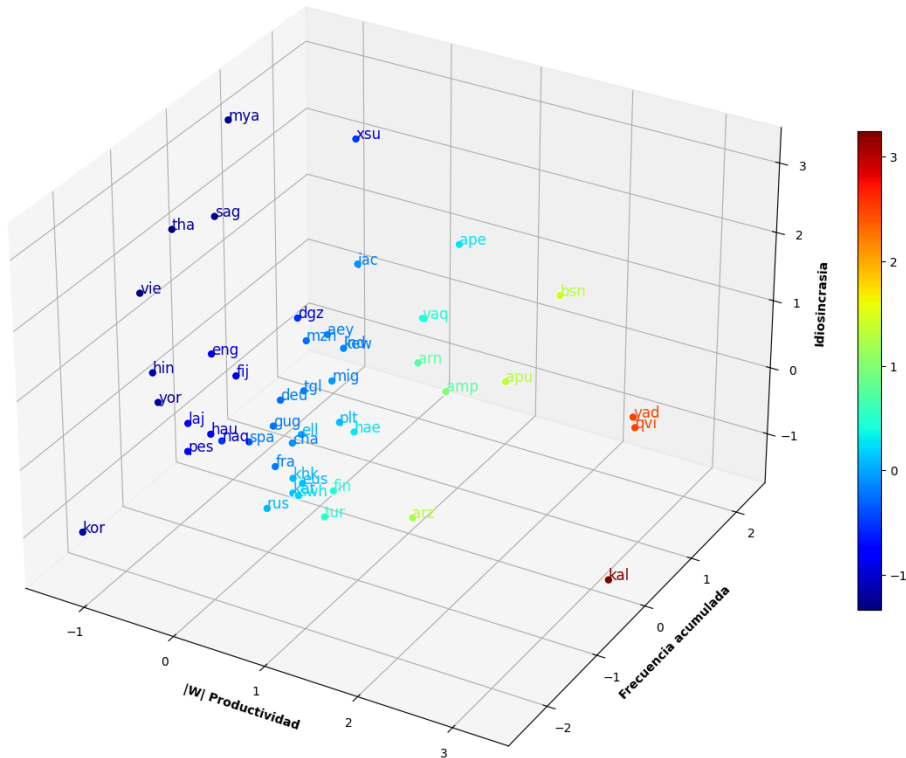


Figura 1: Espacio de BPE definido por Gutierrez-Vasques et al. (2023).

2.4 Bases de datos lingüísticas

2.4.1 WALS

WALS (Dryer & Haspelmath, 2013) es una base de datos que contiene información sobre las propiedades fonológicas, gramaticales y léxicas de hasta 2,662 lenguas y dialectos. Dichas propiedades se encuentran organizadas en hasta 192 características por lengua.

Las características de WALS toman un valor entero positivo y no tienen la misma distribución entre todas estas características. Esto implica que cada valor contiene un significado diferente que varía de acuerdo a cada característica. Por ejemplo, la característica 20A puede tomar 7 valores, 28A puede tomar hasta 4 y 49A hasta 9 valores diferentes.

2.4.2 Grambank

Grambank (Skirgård et al., 2023) es otra base de datos lingüística que registra hasta 195 características de 2,467 lenguas y dialectos en el mundo.

Las características de Grambank en su mayoría son binarias. Así, toman los valores de 0 y 1 (0/no, 1/sí), algo que contrasta al rango de valores que toman las características de WALS. De acuerdo a Haynie et al. (2023), el uso de características binarias permitió evitar ambigüedades en la categorización de las características y permitió registrar los rasgos en términos de presencia o ausencia, en vez de categorizar sólo la más dominante. Sin embargo, no todas las características tienen asignado un valor en algunas lenguas, por lo cual toman un valor de desconocido (?/desconocido). Por ejemplo, considérese la característica GB020:

1. Codifique con 1 si existe un morfema que pueda marcar definitud o especificidad sin transmitir también un significado de íctico espacial.
2. Codifique con 0 si la fuente no menciona un artículo definido y no es posible encontrar uno en los ejemplos o textos de una gramática que, por lo demás, es exhaustiva.
3. Codifique con ? si la gramática no contiene suficiente análisis para determinar si existe o no un artículo definido.
4. Si ha codificado 1 para GB020 y 0 para GB021 y GB022, por favor escriba un comentario explicando la posición del artículo definido o específico.

2.4.3 Lang2vec

Otro conjunto de datos utilizados es lang2vec, que entre sus datos recopila características sintácticas de las lenguas. Estas características vienen de URIEL (Littell et al., 2017) y de características aprendidas (Malaviya et al., 2017) que llenan los valores faltantes. Por lo cual, lang2vec proporciona otro punto de comparación en contraste a las características morfológicas.

2.5 Relación entre espacios

2.5.1 Índice Rand

2.5.2 Índice Rand Ajustado

Se realizó el calculo del Índice Rand Ajustado (*Adjusted Rand Index*, ARI)(Hubert & Arabie, 1985) para observar una relación entre los espacios. La relación mediante el ARI se da a ver en qué tan parecidos son los agrupamientos que se generan en un espacio con los de otro espacio.

De esta manera, si un espacio A presenta un agrupamiento $[a, a, b, b, c, c]$ y un espacio B presenta $[c, c, a, a, b, b]$, el ARI indicaría que ambos agrupamientos son idénticos:

$$\text{ARI}([a, a, b, b, c, c], [c, c, a, a, b, b]) = 1 \quad (8)$$

Un valor de ARI igual a 1 indica agrupamientos idénticos, un valor cercano a 0 indica un agrupamiento aleatorio, y un valor negativo indica una concordancia menor a la esperada por azar.

El calculo del ARI (Hubert & Arabie, 1985) sigue la siguiente fórmula²:

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}} \quad (9)$$

donde:

- n_{ij} : elementos comunes entre cluster i y cluster j .
- $a_i = \sum_j n_{ij}$: tamaño del cluster i en la partición U .
- $b_j = \sum_i n_{ij}$: tamaño del cluster j en la partición V .
- n : número total de elementos.

Como se observa en Ecuación 8, ARI es una medida simétrica, lo que proporciona libertad en la disposición de los clusters durante el cálculo y elimina la necesidad de considerar el orden de los agrupamientos al alimentar el algoritmo.

2.5.3 K-medias

Para generar los agrupamientos se utilizó K-medias (*K-means*) (Lloyd, 1982), un algoritmo que agrupa datos en k conjuntos asignando cada punto al centroide más cercano e iterando hasta que los grupos se estabilicen.

El agrupamiento³ se realizó con 4 clústeres. Los centroides iniciales se seleccionaron mediante el método k-means++ (Arthur & Vassilvitskii, 2007), que distribuye las semillas iniciales de forma probabilística para acelerar la convergencia y reducir la sensibilidad a la inicialización. Se fijó una semilla aleatoria para garantizar la reproducibilidad de los resultados.

²Para los experimentos con ARI, se usó `adjusted_rand_score` de `scikit-learn`.

³Se usó `Kmeans` de `scikit-learn`.

Capítulo 3

Metodología

Queremos hacer una relación entre los espacios dados por los vectores generados por BPE, los generados por el subconjunto de características de WALS y por las características de Grambank. Para ello, vamos a procesar estos datos dado un conjunto de lenguas para luego aplicar una serie de métricas que nos permitan relacionarlos.

3.1 Datos/Corpus de entrenamiento multilingües

Para obtener los vectores del espacio de BPE, usamos la metodología propuesta por Gutierrez-Vasques et al. (2023), la cual toma un texto en una lengua y lo transforma en un vector que caracteriza la productividad, idiosincrasia y frecuencia acumulada de dicha lengua mediante las subpalabras generadas por un modelo de BPE. Así, obtuvimos un vector por cada una de las x lenguas definidas en *placeholder*.

Como datos de entrada en este proceso, utilizamos el Corpus Paralelo de la Biblia (*Parallel Bible Corpus*, PBC) (Mayer & Cysouw, 2014). Usando un corpus paralelo, trabajamos con un texto similar a través de las lenguas.

Para el procesamiento, utilizamos la implementación del proceso que estuvo disponible en GitHub. El programa usó Python y utilidades de UNIX. Sin embargo, modificamos ligeramente algunas partes del código debido a que usamos macOS para obtener los vectores. Este proceso fue realizado para cada lengua y comprendió cinco etapas.

La primera etapa consistió en la *tokenización a nivel de palabra*, en la cual el corpus se dividió en palabras ortográficas para establecer la base del procesamiento posterior. De esta manera, las palabras son diferenciables mediante espacios, distinción que resulta útil para lenguas como el japonés.

Posteriormente, se realizó el *preprocesamiento del corpus* con el objetivo de obtener un mejor modelo de BPE. Este preprocesamiento implicó dos operaciones sobre el texto. En primer lugar, todos los caracteres fueron transformados a minúsculas. Si bien se tuvo consciencia de que en algunas lenguas la relación mayúscula-minúscula no está definida de la misma manera que en la función `lower()` de Python, por razones de reproducibilidad se decidió mantener este criterio. En segundo lugar, se removieron del texto los signos de puntuación `_ . , " ( ) ? ¿ ? ¡ ! » « " " , / \ ] _`. Un ejemplo del preprocesamiento es el siguiente:

Hola, ¿cómo estás? → hola como estás

A continuación, se procedió a la *generación del modelo BPE* a partir del texto preprocesado. El programa utilizado fue `subword-nmt`, configurado con 200 merges. Este número es sugerido como un punto de inflexión donde la entropía de las lenguas es menos dispersa (Gutierrez-Vasques et al., 2021).

La cuarta etapa correspondió a la *obtención de las métricas por subpalabra*. Para ello, se aplicó el modelo BPE entrenado al archivo del corpus preprocesado utilizando `subword-nmt`, tras lo cual se calcularon las medidas de productividad, frecuencia acumulada e idiosincrasia de cada subpalabra.

Finalmente, se llevó a cabo la *obtención de las métricas por lengua*, promediando las métricas obtenidas de cada subpalabra para obtener la representación vectorial de cada lengua.

El resultado final fue una matriz $X \in \mathbb{R}^{n \times 3}$, el cual llamamos espacio BPE. Normalizamos este espacio para la aplicación de los otros métodos:

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j} \quad (10)$$

3.2 Procesamiento computacional de las bases de datos lingüísticas

Con el objetivo de identificar similitudes y correlaciones con el espacio de BPE, se procesaron dos bases de datos lingüísticas: *World Atlas of Language Structures* (WALS) y Grambank. Estas bases de datos contienen características tipológicas —morfológicas, sintácticas y fonológicas— de las lenguas analizadas, las cuales permiten corroborar posibles similitudes con las características morfológicas presentes en el espacio BPE. Sin embargo, ninguna de las dos bases de datos presenta una correspondencia uno a uno entre las lenguas que contienen, y no todas las características cuentan con un valor asignado. Por ello, fue necesario aplicar una serie de procesamientos previos antes de integrarlas al análisis junto con el espacio de BPE.

Dicho procesamiento fue posible dado que tanto Grambank como WALS siguen los *Cross-Linguistic Data Formats* (CLDF) (Forkel et al., 2018), un conjunto de estándares para estructurar, compartir y reutilizar datos lingüísticos. A partir de estas, se extrajeron las características lingüísticas de cada lengua, obteniendo una matriz $X_{\text{Grambank}} \in \mathbb{R}^{n \times m}$ y una matriz $X_{\text{WALS}} \in \mathbb{R}^{n \times m}$, denominadas espacio de Grambank y espacio de WALS, respectivamente.

3.2.1 WALS

Se utilizó un subconjunto de 15 características de WALS que codifican información de tipología morfológica (Gutierrez-Vasques et al., 2023) (véase Tabla 1). Dicho subconjunto presenta una cantidad reducida de valores vacíos para las lenguas analizadas.

Para identificar las lenguas en WALS se utilizó el *WALS code*, ya que el ISO 639-3 puede ser compartido por varias lenguas, lo que dificultaría su distinción. No obstante, el ISO 639-3 se empleó como apoyo para mejorar la identificación de las lenguas.

De WALS, la información de interés fue el nombre de las lenguas y el valor de cada una de sus características⁴. CLDF organiza esta información en tres componentes: las *lenguas*, que son los objetos de investigación; los *parámetros*, que representan los conceptos comparativos medidos entre lenguas y que en este estudio se denominarán características; y los *valores*, que corresponden a la medición concreta de una característica para una lengua específica.

A partir de la base de datos de WALS⁵, se procesó `ValueTable`, que contiene los valores de las características para cada lengua, para construir las representaciones vectoriales de las lenguas, convirtiendo cada una en un vector a partir de dichos valores. Por ejemplo, el inglés con las características de Tabla 1 produce el vector $v = (1, 2, 2, 2, 2, 2, 1, 2, 1, 2, 2, 5, 1, 2, 2)$.

⁴Se obtuvo los datos de WALS de su repositorio público <https://github.com/cldf-datasets/wals>.

⁵A partir del repositorio de WALS, se construyó una base de datos relacional mediante `pycldf` (Forkel et al., 2018). El uso de bases de datos relacionales, frente a otras modalidades disponibles como archivos `.csv` o llamadas a bibliotecas, proporcionó la flexibilidad necesaria para realizar consultas mediante SQL.

Rasgo	Nombre
20A	Fusión de formativos flexivos seleccionados
22A	Síntesis flexiva
26A	Prefijación vs. sufijación en la morfología flexiva
28A	Sincretismo de caso
29A	Sincretismo en la marcación de persona/número verbal
49A	Número de casos
59A	Clasificación posesiva
65A	Aspecto perfectivo/imperfectivo
66A	El tiempo pasado
67A	El tiempo futuro
69A	Posición de los afijos de tiempo/aspecto
70A	El imperativo morfológico
78A	Codificación de la evidencialidad
102A	Marcación de persona verbal
112A	Morfemas negativos

Tabla 1: Tabla de rasgos de WALS usados por para describir tipología morfológica (Gutierrez-Vasques et al., 2023)

Los vectores de las lenguas se agruparon en una matriz $X_{\text{WALS}} \in \mathbb{R}^{n \times m}$, donde n es el número de lenguas y m el número de características. Durante este procesamiento, se identificó que algunas lenguas carecen de valores para ciertas características. Dichos valores se dejaron como 0 en la matriz.

3.2.2 Grambank

A pesar de las similitudes entre Grambank y WALS, las lenguas presentes en ambas bases de datos no presentan una correspondencia uno a uno. Si bien algunas lenguas pueden relacionarse por nombre, como el inglés, en otros casos la relación es más compleja: una lengua en WALS puede corresponder a múltiples entradas en Grambank, y viceversa. Esta complejidad se acentúa debido a que Grambank utiliza identificadores propios y no el ISO 639-3, lo que dificulta aún más establecer una correspondencia entre ambas bases de datos.

Por consiguiente, para relacionar las lenguas de Grambank con las de WALS, se establecieron tres criterios de selección, priorizando siempre las lenguas con más características disponibles en Grambank.

El primer criterio fue la coincidencia exacta de nombre, relacionando directamente las lenguas que compartían el mismo nombre en ambas bases de datos, como *Modern Greek* para el griego. El segundo criterio aplicó cuando solo existía una lengua con nombre similar, considerándola como equivalente; por ejemplo, *Lango (Uganda)* en Grambank correspondió a *Lango* en WALS. Finalmente, si existían múltiples lenguas con nombre similar, se eligió la que contara con más características disponibles; por ejemplo, *Hausa States Fulfulde* se seleccionó sobre *Hausa* por contar con más características en Grambank.

Cabe señalar que Grambank no cuenta con información de todas las lenguas de interés, como el español y el alemán, por lo que el conjunto de lenguas analizado se redujo en consecuencia.

La matriz X_{Grambank} se construyó mediante el mismo método que X_{WALS} , leyendo los datos a través de CLDF⁶. Cabe resaltar que, a diferencia de WALS, se utilizó el identificador propio de Grambank para identificar las lenguas.

Sin embargo, la selección de características de Grambank requirió una exploración previa, ya que no todas las lenguas cuentan con el mismo conjunto de características disponibles. Por ello, se priorizó la combinación de características que minimizara los valores vacíos, ordenándolas de mayor a menor según el número de lenguas que cubrían. Por ejemplo, GB107 cubre todas las lenguas y tendría alta prioridad, mientras que GB401 cubre pocas lenguas y se seleccionaría en los últimos lugares.

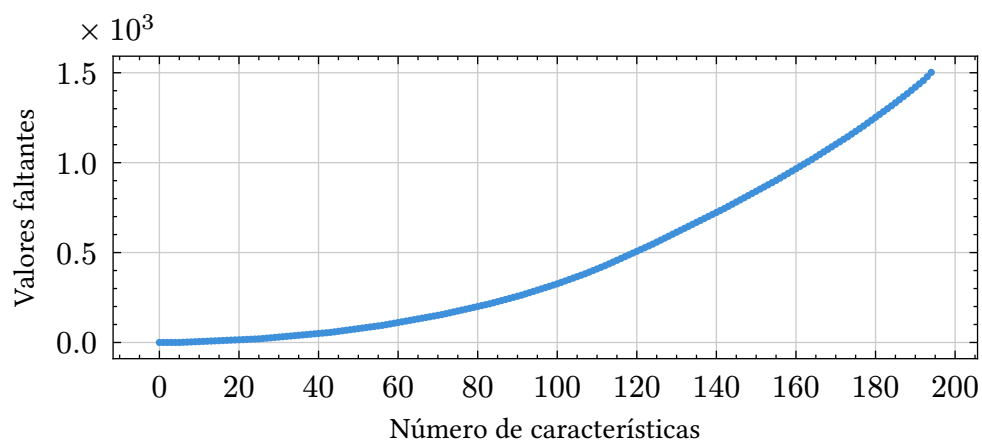


Figura 2: Número de valores faltantes al ir agregando más características.

Como se observa en Figura 2, a partir de las 80 características la tendencia de valores vacíos incrementa. Un análisis más detallado por lengua revela que algunas, como el barasano y el oromo, presentan una gran cantidad de valores vacíos, lo cual contribuye a este incremento.

No obstante, dado que la mayoría de las lenguas cuentan con un promedio aceptable de valores vacíos, no se descartó ninguna, con el fin de abarcar la mayor cantidad de lenguas posible.

Al igual que con WALS, los algoritmos requieren que la matriz de Grambank no contenga valores nulos, por lo que fue necesario imputar los valores faltantes. Para ello, se consideraron cuatro estrategias de imputación: reemplazar los valores nulos con 0, asumiendo su ausencia; reemplazarlos con -1 , modelando la ausencia como un valor desconocido; imputar la media del conjunto, basándose en los valores de las demás lenguas de la matriz; o imputar valores de lenguas cercanas, lo que proporcionaría una aproximación más informada, aunque esta opción se descartó por requerir conocimiento lingüístico especializado.

La decisión fue imputar con 0 los valores vacíos en X_{Grambank} , para interpretarlo como la ausencia de esta feature. Esta interpretación concuerda en la mayoría de las features de Grambank, que son binarias.

Otra paso del procesamiento fue estandarizar y centrar los puntos que obtuvimos después de construir la matriz. Esto se logra con:

⁶Se obtuvo los datos de Grambank de su repositorio público <https://github.com/grambank/grambank>.

3.2.3 lang2vec

Las características sintácticas empleadas provienen de dos conjuntos: `syntax_wals`, basado directamente en WALS, y `syntax_knn`, que aplica una técnica de k vecinos más cercanos sobre la combinación de WALS y otras bases de datos. Ambos conjuntos cubren las lenguas del estudio; sin embargo, `syntax_wals` puede contener valores vacíos para algunas características, mientras que `syntax_knn` no presenta ninguno valor vacío por las características aprendidas.

Los valores faltantes en `syntax_wals`, representados como `--`, se imputaron con 0 para denotar la ausencia de un valor, de manera consistente con el criterio adoptado para Grambank. Esta decisión resulta apropiada dado que las características de `lang2vec` son binarias (Littell et al., 2017).

En cuanto a la cobertura de lenguas, `lang2vec` incluye todas las descritas en (placeholder), identificándolas mediante sus códigos ISO.

3.3 Base de referencia

En adición a los anteriores conjuntos de vectores, creamos un espacio aleatorio basado en el espacio BPE que sirvió como una base de referencia. Con esta base, podemos establecer si hay una mayor relación entre el espacio de BPE y Grambank o WALS más allá del carácter aleatorio.

Por tal motivo, para crear la base de referencia, obtuvimos los rangos donde varían las características de BPE. Con esta información, generamos una distribución uniforme por cada característica para crear los vectores para cada lengua en este nuevo espacio. A este espacio lo denominamos $X_{\text{BPE Random}}$.

3.4 Comparación entre los espacios

Para calcular la similitud entre un espacio X y un espacio Y , se utilizó el ARI sobre los agrupamientos de cada espacio, obtenidos mediante `k-medias++`. Este algoritmo depende de una semilla de inicialización, por lo que los resultados pueden variar entre ejecuciones. Para dar cuenta de esta variabilidad, se evaluaron todas las permutaciones ordenadas de pares de semillas $s_1, s_2 \in \{0, 1, \dots, 99\}$. Estas permutaciones generaron un total de 10,000 valores de ARI para cada par de espacios X, Y .

En todos los experimentos que involucran X_{BPE} , se calculó también el ARI con $X_{\text{BPE-r}}$ como marco de referencia.

3.4.1 Experimentos

3.4.2 BPE vs WALS+Grambank

El objetivo de este experimento fue comprobar si Grambank, como una extensión de WALS, ayuda a mejorar los valores de relación con BPE.

Para realizar esto, se construyó $X_{\text{WALS+Grambank}}$ mediante la concatenación horizontal de X_{WALS} y X_{Grambank} , es decir, $X_{\text{WALS+Grambank}} = [X_{\text{WALS}} \mid X_{\text{Grambank}}]$. Como Grambank ofrece un menor soporte de lenguas, se tuvo que reducir el número de lenguas en WALS para que coincidieran con las de Grambank.

Esto resultó en un espacio $X_{\text{Grambank+WALS}}$ de dimensión $38 \times (m + n)$, donde m es el número de características fijas definidas en Tabla 1 y n es el número de características variable según la selección descrita en la sección anterior. Dado que el experimento se corrió por cada valor de n , se obtuvieron en total $10,000 \times n$ valores ARI.

3.4.3 BPE vs WALS

Para evaluar la similitud entre X_{BPE} y X_{WALS} , se siguió la configuración de lenguas y características de WALS establecida por Gutierrez-Vasques et al. (2023). A diferencia del experimento anterior, aquí se conservó el conjunto completo de lenguas, sin la reducción impuesta por Grambank.

3.4.4 BPE vs Grambank

El objetivo de este experimento fue evaluar la relación entre X_{BPE} y X_{Grambank} de manera aislada. Para esto, se utilizó X_{Grambank} de dimensión $38 \times n$, donde n es el número de características según la selección descrita en la sección anterior. Dado que el experimento se corrió por cada valor de n , se obtuvieron en total $10,000 \times n$ valores de ARI.

3.4.5 Grambank vs WALS

El objetivo de este experimento fue evaluar la relación entre X_{BPE} y X_{Grambank} de manera aislada. Para esto, se utilizó X_{Grambank} de dimensión $38 \times n$, donde n es el número de características según la selección descrita en la sección anterior. Dado que el experimento se corrió por cada valor de n , se obtuvieron en total $10,000 \times n$ valores de ARI.

3.4.6 Grambank vs lang2Vec

Se calculó el ARI entre X_{Grambank} y de X_{lang2vec} para evaluar qué tan similares son los agrupamientos cuando uno de los espacios se basa en características sintácticas. De lang2vec, se utilizaron los conjuntos `syntax_wals` y `syntax_knn`, con un número fijo de características m . Para Grambank, el número de características n se mantuvo variable como en los experimentos anteriores, resultando en $10,000 \times n$ valores de ARI.

Capítulo 4

Resultados

4.1 BPE vs WALS+Grambank

La Figura 3 representa cómo están distribuidos los valores de ARI de X_{BPE} y X_{Grambank} en base al criterio de selección de características. El número de características se encuentra entre 0 y 195. La Figura 7 representa de manera similar lo que la figura anterior pero sustituyendo X_{BPE} por $X_{\text{BPE-T}}$. Por lo que Figura 4 representa la base de referencia.

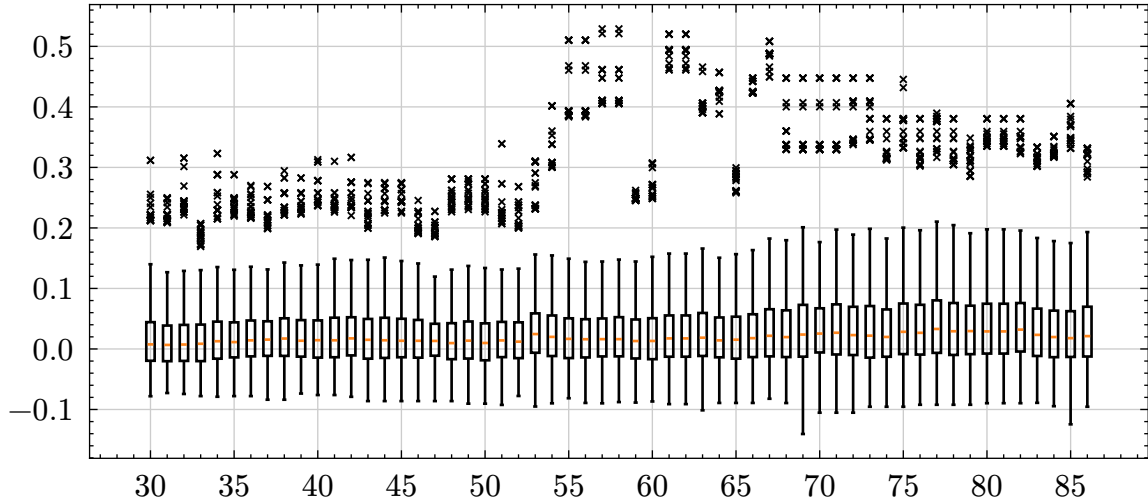


Figura 3: Resultados de los valores de ARI usando syntax_wals.

Como se observa en Figura 3, el valor de las medianas de ARI está encima del 0. Se observa además que el rango de la mayoría de las configuraciones es entre -0.1 y 0.2 . Además, se tienen valores outliers que superan el umbral de 0.5 . Esto se puede observar en $n = 55, n = 56, n = 57, 58, 61, 62, 67$.

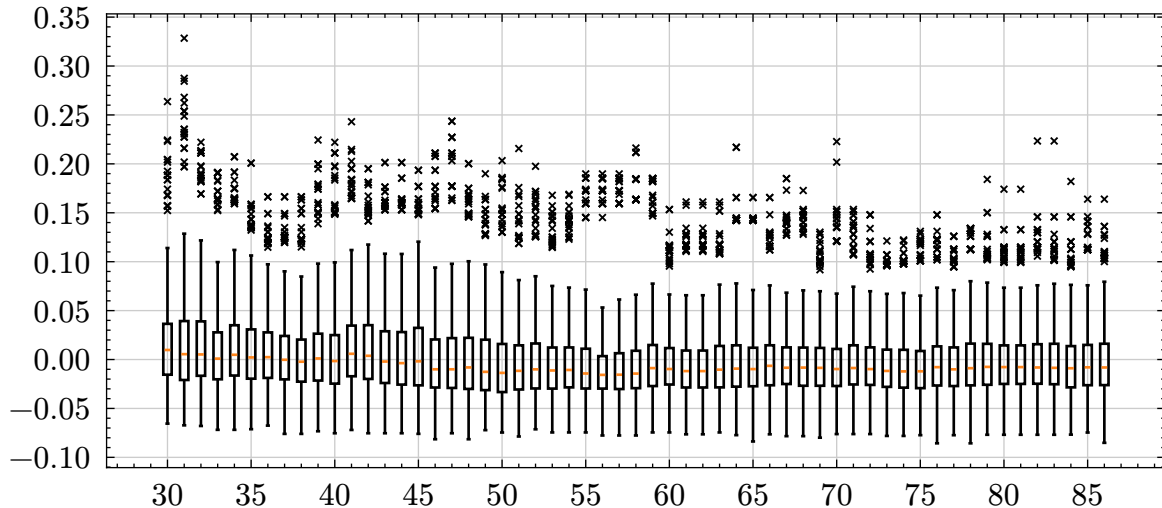


Figura 4: Resultados de los valores de ARI usando syntax_knn.

En contraste, Figura 4, se observa que a partir de $n = 43$, el valor de la medianas se encuentran abajo del 0. En contraste con Figura 3, sólo hay un outlier que supera el umbral de 0.3 , y es cuando $n = 31$, mientras que en n más grandes, esto no se vuelve a repetir. Si bien hasta

$n = 48$ algunos de los valores (o pestaña) superan el umbra del 0.1, a partir de ahí los valores varían entre 0.1 y 0.5.

4.2 BPE vs WALS

La Figura 5 representa cómo están distribuidos los valores de ARI calculados para X_{BPE} y X_{WALS} (izquierda), como para $X_{\text{BPE-r}}$ y X_{WALS} (derecha) que representa la base de referencia.

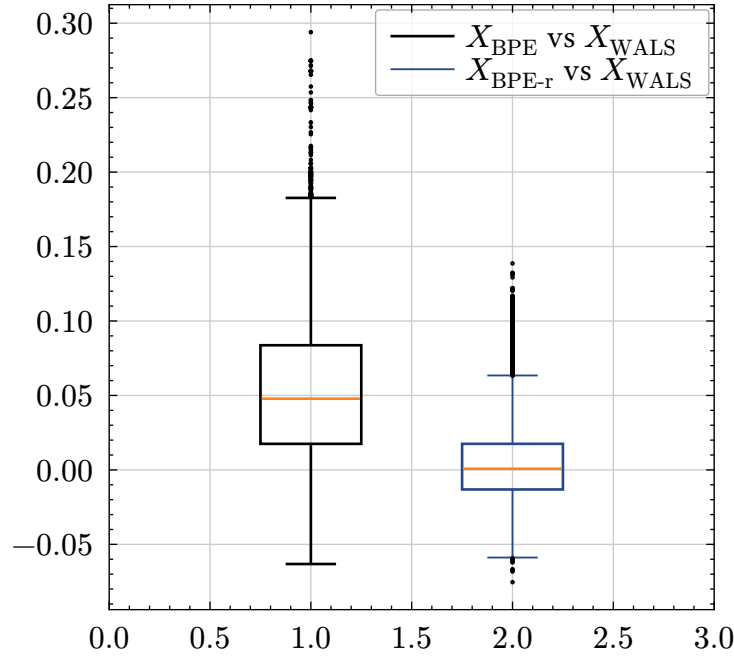


Figura 5: Resultados de los valores de ARI.

Como se observa los datos ARI de X_{BPE} y X_{WALS} , el 50% de los valores ARI se encuentran entre los rangos 0.0175 y 0.0837. El rango donde varían estos valores es entre -0.0631 y 0.1826 . Hay que notar que Figura 5 tiene varios outliers, donde varios superan en valor de 0.20. La media tiene un valor 0.0478. El máximo valor que alcanzó es 0.2939 y el mínimo -0.0631 .

Sin embargo, para $X_{\text{BPE-r}}$ y X_{WALS} , el 50% de los datos se encuentran contenidos en un rango menor al anterior, pues está entre -0.0131 y 0.0175 . A la vez, el rango de datos se encuentra entre -0.0591 y 0.0634 . Por otro lado, los outliers, pero no superaron 0.15. La media es 0.0007. El máximo valor que alcanzó es 0.1386 y el mínimo -0.0753 .

4.3 BPE vs Grambank

La Figura 6 representa cómo están distribuidos los valores de ARI de X_{BPE} y X_{Grambank} en base al criterio de selección de características. El número de características se encuentra entre 0 y 195. La Figura 7 representa de manera similar lo que la figura anterior pero sustituyendo X_{BPE} por $X_{\text{BPE-r}}$. Por lo que Figura 7 representa la base de referencia.

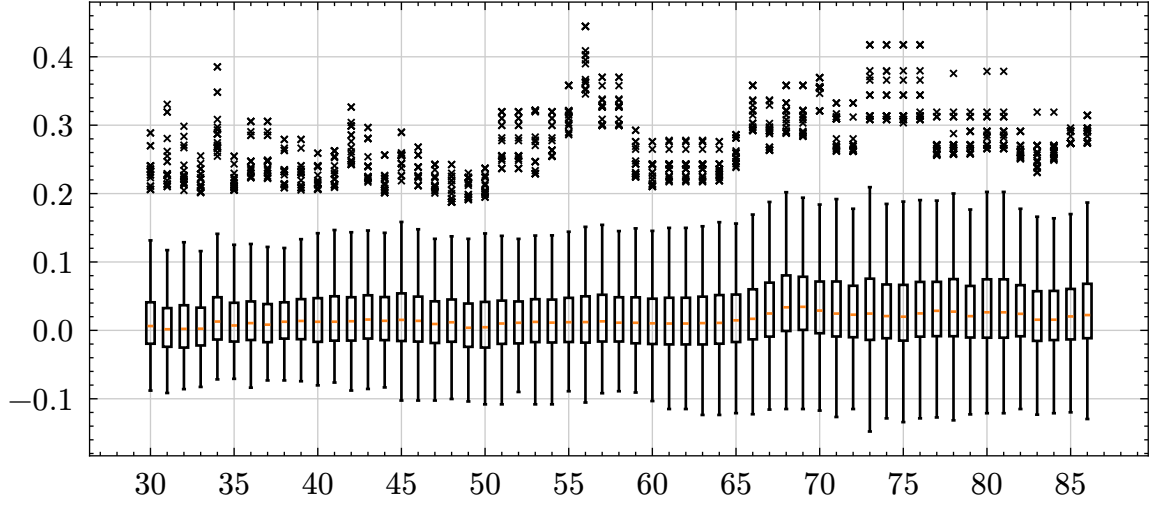


Figura 6: Resultados de los valores de ARI de BPE vs Grambank.

Se observa en Figura 6 que la mayoría de los valores independientemente del número de características se distribuyen entre los rangos de -0.1 y 0.2 , con algunas excepciones después de $n = .65$. Se observa además que la mediana no aumenta drásticamente al incrementar el número de características. Podemos observar varios outliers que superan el umbral de 0.4 , como en $n = 56$, así como $n = 73, 74, 75, 76$. El valor de las medias se mantiene positivo. El bigote inferior pueden superar el umbral de -0.1 , lo cual indica que hay más variabilidad de datos en esos n .

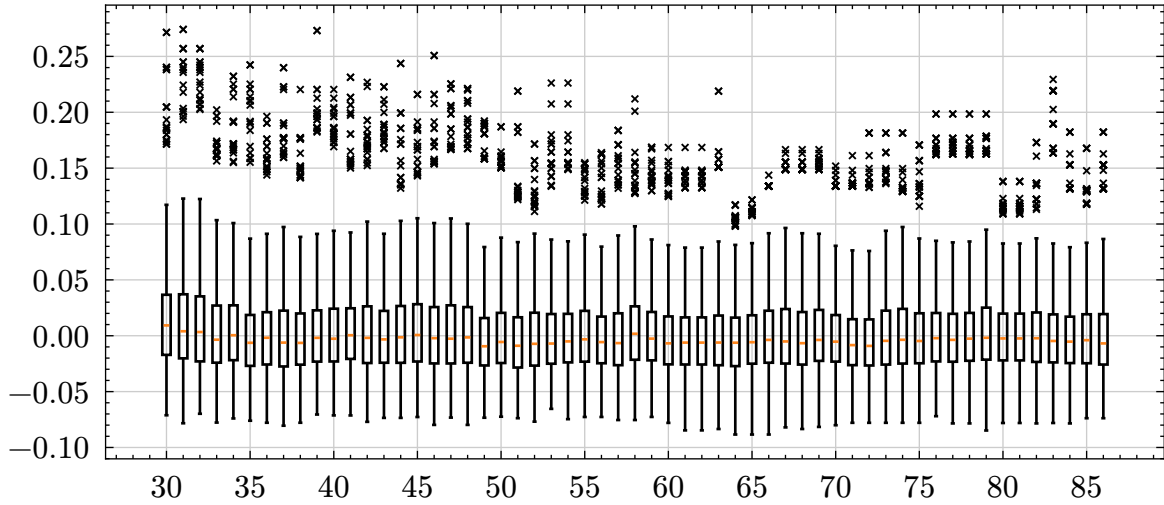


Figura 7: Resultados de los valores de ARI de BPE Aleatorio vs Grambank.

Por otra parte, se observa en Figura 7 que son pocos los outliers que cruzan el umbral de 0.25 , que se encuentran antes de $n = 50$. De la misma manera, la mediana de ARI en muchos casos es negativa. Se observan que el bigote superior de la mayoría de las n no superan el umbral de 0.1 .

4.4 Grambank vs WALS

La Figura 8 representa cómo están distribuidos los valores de ARI de X_{Grambank} y X_{WALS} en base al criterio de selección de características. El número de características se encuentra entre 0 y 195 .

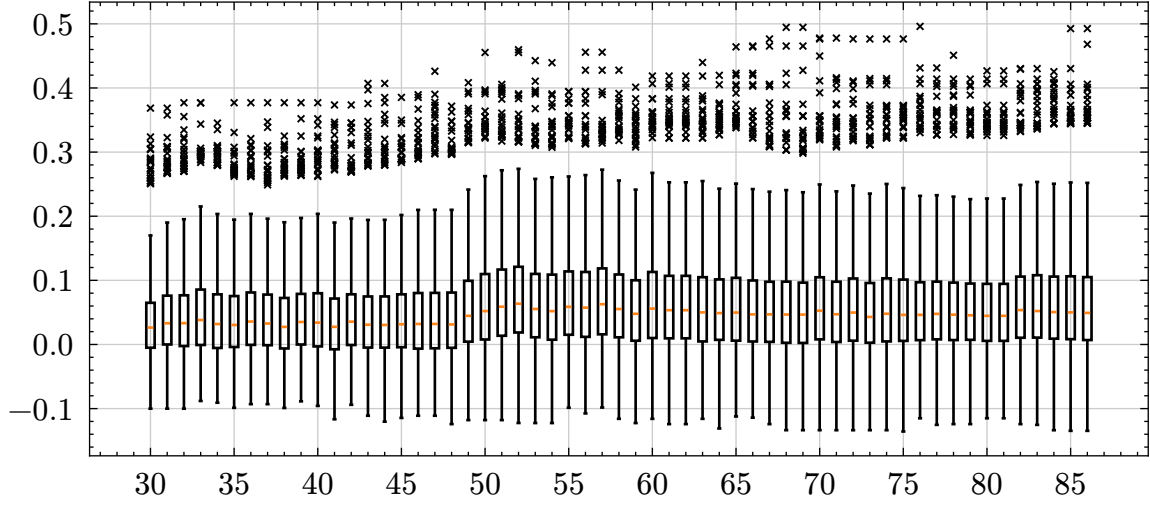


Figura 8: Resultados de los valores de ARI.

Se observa en Figura 8 varios outliers entre 0.4 y 0.5. Se puede observar que las medianas son positivas, variando a un valor cercano a 0.05. Por otro lado, se observe que a partir de $n = 45$, el bigote superior se encuentra encima del umbral de 0.2. Sin embargo, también hay una constante en los bigotes inferiores que suelen superar el valor de -0.1 .

4.5 Grambank vs Lang2Vec

La Figura 9 representa cómo están distribuidos los valores de ARI de X_{Grambank} y X_{lang2vec} con `syntax_wals`, en base al criterio de selección de características. El número de características se encuentra entre 0 y 195. Mientras que Figura 10 representa lo mismo pero usando el conjunto `syntax_knn`.

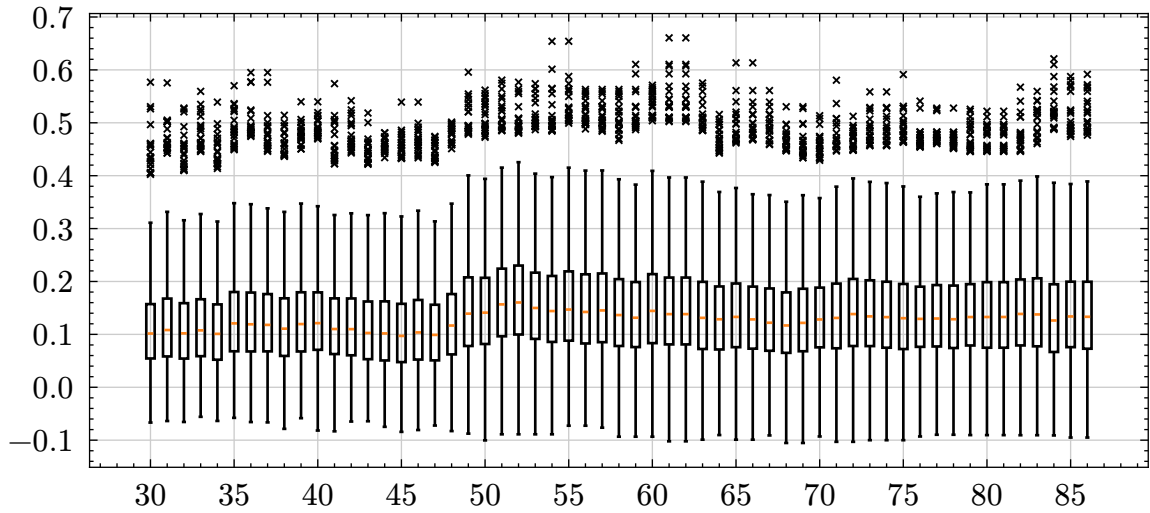


Figura 9: Resultados de los valores de ARI usando `syntax_wals`.

Se observa en Figura 9 que las medias superan el umbral del 0.1. A su vez, los bigotes inferiores son menores que 0.1. Hay varios bigotes superiores que se encuentran cerca a 0.4. Referente a los outliers, hay varios que superan el umbral de 0.6, como en $n = 54, 55, n = 61, 62, 65, 66, 84$.

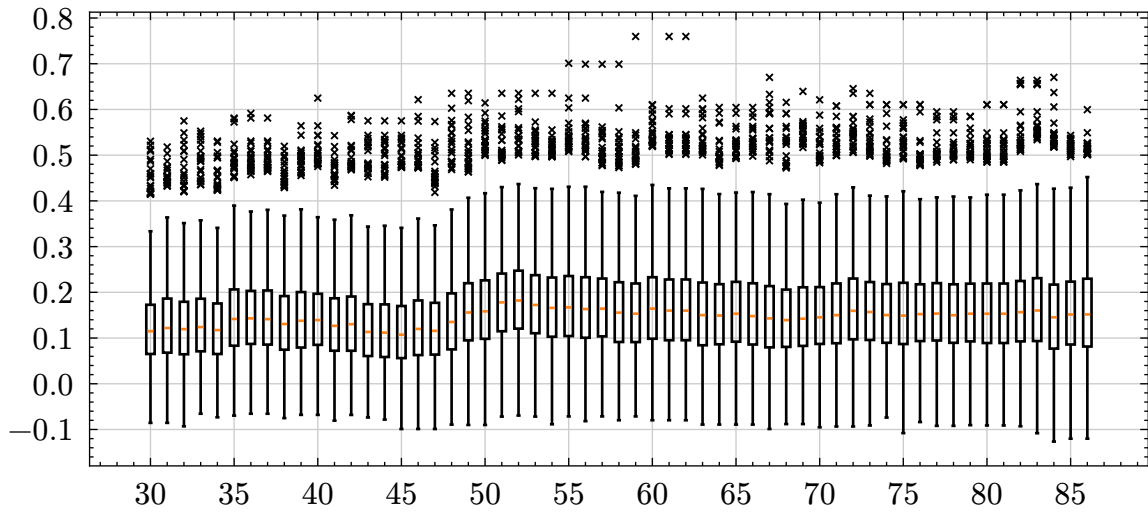


Figura 10: Resultados de los valores de ARI usando `syntax_knn`.

Se observa en Figura 10, hay outliers que toman el valor de 0.7 o más. Se puede observar esto en $n = 55, 56, 57, 58, 61, 62$. También hay mucha variabilidad de los datos, como siendo los bigotes inferiores cercanos a -0.1 , mientras que los superiores están por encima de 0.4 a partir de $n = 50$. Las medianas están por encima de 0.1.

Capítulo 5

Discusión

Capítulo 6

Conclusiones

Bibliografía

- Arthur, D., & Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-1035.
- Bayram, M. A., Fincan, A. A., Gümüş, A. S., Karakaş, S., Diri, B., & Yıldırım, S. (2025,). *Tokenization Standards for Linguistic Integrity: Turkish as a Benchmark*. <https://arxiv.org/abs/2502.07057>
- Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2003). A neural probabilistic language model. *J. Mach. Learn. Res.*, 3(null), 1137-1155.
- Brown, P. F., Cocke, J., Della Pietra, S. A., Della Pietra, V. J., Jelinek, F., Lafferty, J. D., Mercer, R. L., & Roossin, P. S. (1990). A Statistical Approach to Machine Translation. *Computational Linguistics*, 16(2), 79-85. <https://aclanthology.org/J90-2002/>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. En J. Burstein, C. Doran, & T. Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers): Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. <https://doi.org/10.18653/v1/N19-1423>
- Domingo, M., Garcia-Martinez, M., Helle, A., Casacuberta, F., & Herranz, M. (2019,). *How Much Does Tokenization Affect Neural Machine Translation?*. <https://arxiv.org/abs/1812.08621>
- Dryer, M. S., & Haspelmath, M. (eds.). (2013). *WALS Online (v2020.4)* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.13950591>
- Forkel, R., List, J.-M., Greenhill, S. J., Rzymiski, C., Bank, S., Cysouw, M., Hammarström, H., Haspelmath, M., Kaiping, G. A., & Gray, R. D. (2018). Cross-Linguistic Data Formats, advancing data sharing and re-use in comparative linguistics. *Scientific Data*, 5(1), 180205. <https://doi.org/10.1038/sdata.2018.205>
- Gage, P. (1994). A new algorithm for data compression. *The C Users Journal*, 12(2), 23-38.
- Gutierrez-Vasques, X., Bentz, C., & Samardžić, T. (2023). Languages Through the Looking Glass of BPE Compression. *Computational Linguistics*, 49(4), 943-1001. https://doi.org/10.1162/coli_a_00489
- Gutierrez-Vasques, X., Bentz, C., Sozinova, O., & Samardzic, T. (2021). From characters to words: the turning point of BPE merges. En P. Merlo, J. Tiedemann, & R. Tsarfaty (eds.), *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. <https://doi.org/10.18653/v1/2021.eacl-main.302>
- Haynie, H. J., Blasi, D., Skirgård, H., Greenhill, S. J., Atkinson, Q. D., & Gray, R. D. (2023). Grambank's Typological Advances Support Computational Research on Diverse Languages. En L. Beinborn, K. Goswami, S. Muradoğlu, A. Sorokin, R. Kumar, A. Shcherbakov, E.

- M. Ponti, R. Cotterell, & E. Vylomova (eds.), *Proceedings of the 5th Workshop on Research in Computational Linguistic Typology and Multilingual NLP: Proceedings of the 5th Workshop on Research in Computational Linguistic Typology and Multilingual NLP*. <https://doi.org/10.18653/v1/2023.sigtyp-1.17>
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193-218. <https://doi.org/10.1007/BF01908075>
- Hutchins, W. J. (2004). The Georgetown-IBM experiment demonstrated in January 1954. En R. E. Frederking & K. B. Taylor (eds.), *Proceedings of the 6th Conference of the Association for Machine Translation in the Americas: Technical Papers: Proceedings of the 6th Conference of the Association for Machine Translation in the Americas: Technical Papers*. <https://aclanthology.org/2004.amta-papers.12/>
- Johnson, R., King, M., & Tombe, L. des. (1985). EUROTRA: A Multilingual System under Development. *Computational Linguistics*, 11(2-3), 155-169. <https://aclanthology.org/J85-2005/>
- Jurafsky, D., & Martin, J. H. (2025). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models* (3rd ed.). <https://web.stanford.edu/~jurafsky/slp3/>
- Littell, P., Mortensen, D. R., Lin, K., Kairis, K., Turner, C., & Levin, L. (2017). URIEL and lang2vec: Representing languages as typological, geographical, and phylogenetic vectors. *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, 8-14. <http://aclweb.org/anthology/E17-2002>
- Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137. <https://doi.org/10.1109/TIT.1982.1056489>
- Malaviya, C., Neubig, G., & Littell, P. (2017, septiembre). Learning Language Representations for Typology Prediction. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Markov, A. A. (2006). An Example of Statistical Investigation of the Text Eugene Onegin Concerning the Connection of Samples in Chains. *Science in Context*, 19(4), 591-600. <https://doi.org/10.1017/S0269889706001074>
- Mayer, T., & Cysouw, M. (2014). Creating a massively parallel Bible corpus. En N. Calzolari, K. Choukri, T. Declerck, H. Loftsson, B. Maegaard, J. Mariani, A. Moreno, J. Odiijk, & S. Piperidis (eds.), *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14): Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*. <https://aclanthology.org/L14-1215/>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013,). *Efficient Estimation of Word Representations in Vector Space*. <https://arxiv.org/abs/1301.3781>
- Parra, I. (2024,). *Morphological Typology in BPE Subword Productivity and Language Modeling*. <https://arxiv.org/abs/2410.23656>

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. En K. Erk & N. A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers): Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. <https://doi.org/10.18653/v1/P16-1162>
- Skirgård, H., Haynie, H. J., Hammarström, H., Blasi, D. E., Collins, J., Lata arche, J., Lesage, J., Weber, T., Witzlack-Makarevich, A., Dunn, M., Reesink, G., Singer, R., Bower n, C., Epps, P., Hill, J., Vesakoski, O., Abbas, N. K., Ananth, S., Auer, D., ... Gray, R. D. (2023, marzo). *Grambank v1.0*. Zenodo. <https://doi.org/10.5281/zenodo.7740140>
- Varile, G. B., & Lau, P. (1988). Eurotra Practical Experience With a Multilingual Machine Translation System Under Development. *Second Conference on Applied Natural Language Processing*, 160-167. <https://doi.org/10.3115/974235.974264>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *CoRR*. <http://arxiv.org/abs/1706.03762>
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., ... Wen, J.-R. (2025,). *A Survey of Large Language Models*. <https://arxiv.org/abs/2303.18223>

7 Apéndice

7.1 Conjunto de lenguas utilizadas

Lengua (Código ISO)	Grambank ID	WALS ID
cha	cham1312	cha
gug	mbya1239	gua
fra	stan1290	fre
deu		ger
spa		spa
naq	nama1264	kho
hau	nige1253	hau
pes	west2369	prs
fij	fji1243	fij
laj	lang1324	lan
kor	kore1280	kor
eng	stan1293	eng
hin	hind1269	hin
yor		yor
vie	viet1252	vie
tha	thai1261	tha
mig	sanm1295	mxg
tgl	taga1270	tag
jac	popt1235	jak
aey		ame
kew	east2516	kew
ind	indo1316	ind
mzh		wch
xsu	sanu1240	snm
dgz		dag
sag		san
mya	nucl1310	brm
bsn	bara1380	brs
arn	mapu1245	map
yaq	yaqu1251	yaq
ape	buki1249	arp
yad	yagu1244	yag
qvi	hual1241	qim
apu	apur1254	apu
amp	alam1246	ala
hae	bora1271	orh

Lengua (Código ISO)	Grambank ID	WALS ID
plt	plat1254	mal
kal	kala1399	gre
arz	egyp1253	aeg
tur	nucl1301	tur
fin	finn1318	fin
swh	swah1253	swa
kat	nucl1302	geo
eus	basq1248	bsq
khk	halh1238	kha
rus	russ1263	rus
ell	mode1248	grk

7.2 Conjunto de características utilizadas en Grambank

Rasgo	Nombre
GB107	¿Puede la negación estándar marcarse mediante un afijo, clítico o modificación del verbo?
GB133	¿El orden de constituyentes pragmáticamente no marcado es de verbo final para cláusulas transitivas?
GB025	¿Cuál es el orden del demostrativo adnominal y el sustantivo?
GB065	¿Cuál es el orden pragmáticamente no marcado del sustantivo poseedor adnominal y el sustantivo poseído?
GB093	¿Puede el argumento P ser indexado por un sufijo/enclítico en el verbo en la cláusula principal simple?
GB094	¿Puede el argumento P ser indexado por un prefijo/proclítico en el verbo en la cláusula principal simple?
GB030	¿Existe una distinción de género en los pronombres independientes de 3. ^a persona?
GB028	¿Existe una distinción entre inclusivo y exclusivo?
GB023	¿Existen artículos posnominales?
GB020	¿Existen artículos definidos o específicos?
GB035	¿Existen tres o más contrastes de distancia en los demostrativos?
GB036	¿Los demostrativos muestran una distinción de elevación?
GB147	¿Existe una pasiva morfológica marcada en el verbo léxico?
GB083	¿Existe marcación morfológica explícita en el verbo dedicada al tiempo pasado?
GB084	¿Existe marcación morfológica explícita en el verbo dedicada al tiempo futuro?
GB043	¿Existe marcación morfológica dual productiva en los sustantivos?
GB044	¿Existe marcación morfológica plural productiva en los sustantivos?
GB117	¿Existe una cópula para los predicados nominales?
GB130	¿Cuál es el orden pragmáticamente no marcado de S y V en cláusulas intransitivas?
GB139	¿Existe una diferencia entre las construcciones de negación imperativa (prohibitiva) y declarativa?
GB132	¿El orden de constituyentes pragmáticamente no marcado es de verbo medial para cláusulas transitivas?
GB155	¿Los causativos se forman mediante afijos o clíticos en los verbos?
GB031	¿Existe una forma dual o de aumentativo unitario (además del plural o aumentativo) para todas las categorías de persona en el sistema pronominal?
GB042	¿Existe marcación morfológica singular productiva y explícita en los sustantivos?
GB131	¿El orden de constituyentes pragmáticamente no marcado es de verbo inicial para cláusulas transitivas?
GB021	¿Los sintagmas nominales indefinidos suelen tener artículos indefinidos?
GB022	¿Existen artículos prenominales?
GB057	¿Existen clasificadores numerales?
GB158	¿Se reduplican los verbos?

Rasgo	Nombre
GB024	¿Cuál es el orden del numeral y el sustantivo en el sintagma nominal?
GB038	¿Existen clasificadores demostrativos?
GB047	¿Existe un patrón morfológico productivo para derivar un sustantivo de acción/estado a partir de un verbo?
GB105	¿Puede el receptor en una construcción ditransitiva marcarse como el paciente monotransitivo?
GB135	¿Los objetos clausales suelen aparecer en la misma posición que los objetos nominales?
GB092	¿Puede el argumento A ser indexado por un prefijo/proclítico en el verbo en la cláusula principal simple?
GB090	¿Puede el argumento S ser indexado por un prefijo/proclítico en el verbo en la cláusula principal simple?
GB089	¿Puede el argumento S ser indexado por un sufijo/enclítico en el verbo en la cláusula principal simple?
GB091	¿Puede el argumento A ser indexado por un sufijo/enclítico en el verbo en la cláusula principal simple?
GB049	¿Existe un patrón morfológico productivo para derivar un sustantivo objeto a partir de un verbo?
GB074	¿Existen preposiciones?